

Programação Aplicada

Interfaces Gráficos

Componentes e Gestão de Eventos

Marco Veloso

marco.veloso@estgoh.ipc.pt

(Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino)

Agenda

Interfaces Gráficos – Componentes e Gestão de Eventos

Elementos Gráficos, AWT e Applets

Sessão 0

Interface Gráficos

Componentes de um Interface Gráfico

Sessão 1

Gestão de Eventos

Contentores e Gestores de Posicionamento

Sessão 2

Interacção com o utilizador

Caixas de diálogo e menus

Sessão 3

Elementos Multimédia

Listagens e Tabelas

Sessão 4

Formatações gráficas

Elementos Gráficos, AWT e Applets

Interface Gráficos

Componentes de um Interface Gráfico

Gestão de Eventos

Contentores e Gestores de Posicionamento

Interacção com o utilizador

Caixas de diálogo e menus

Elementos Multimédia

Listagens e Tabelas

Formatações gráficas

Interface Gráficos

Muitas aplicações disponibilizam aos seus utilizadores um interface gráfico (**GUI - *Graphical User Interface***)

A *package* `java.awt` suporta a criação deste tipo de interfaces

Os elementos chave de um interface gráfico em Java são:

- **Componentes**
- **Gestores de posicionamento** (*layout managers*)
- **Processadores de eventos** (*event listeners*)

Componentes são elementos como botões ou campos de texto que podem ser manipulados pelo utilizador com o rato ou o teclado

Gestores de posicionamento determinam a forma como os componentes aparecem no ecrã

Processadores de eventos respondem a acções do utilizadores (ex: pressão num botão do rato)

Os programas com interfaces gráficas devem responder a eventos, gerados pelos componentes, que indicam que determinadas acções ocorreram

Existe uma categoria de classes chamada ***listeners*** (e.g. **ActionListener**) que está atenta à ocorrência de eventos

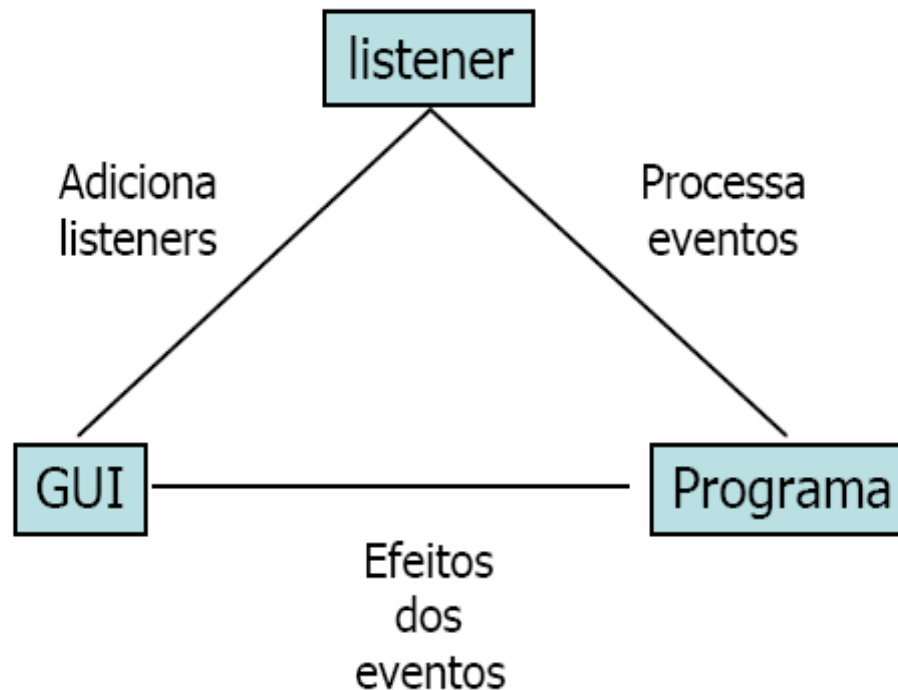
Assim, um programa deste tipo é composto por:

- O código que apresenta a interface ao utilizador
- Os *listeners* que esperam que ocorram eventos
- O código que é executado quando ocorre um evento

Componentes de uma interface gráfica

Interfaces Gráficas – Componentes e Gestão de Eventos

O modelo de programação de interfaces gráficas pode ser representado por:



Assim, no geral a **programação de interfaces gráficas** consiste em:

- **Colocar os componentes** desejados
- **Esperar por uma acção** do utilizador sobre um dos componentes
- Quando a **acção acontece** é detectada pelo *event listener* que chama o(s) método(s) apropriado(s)

Esta técnica é geralmente designada por **programação dirigida por eventos** (*event-driven programming*)

Estas aplicações **baseiam-se geralmente em janelas que contêm os diversos componentes** (que podem ser do tipo **JApplet**, **JFrame**, **JWindow** OU **JPanel**, estudadas mais à frente com detalhe)

Em 1997 foram disponibilizadas as **Java Foundation Classes (JFC)**, as quais incluem um **conjunto de características** que permitem tornar o **desenvolvimento de interfaces gráficas (GUI)** mais flexível e portátil

Entre outras funcionalidades, as JFC incluem a **AWT (*Abstract Window Toolkit*)**, assim como um **conjunto muito mais completo de componentes gráficos** designados por **Swing** (que posteriormente substitui a designação JFC), formando a biblioteca base para desenvolvimento gráfico em Java. Contrariamente às componentes AWT, **Java Swing é independente da plataforma de codificação**

Entre algumas das novas funcionalidades incluem-se **generalizações e melhoramentos de componentes AWT** (e.g., a possibilidade de se definirem botões com imagens), tabelas, *tabbed dialogs*, *tooltips*, árvores, etc.

Em 2005 é introduzido o **Java FX**, incorporado no JSE como biblioteca nativa em 2008 e fazendo parte OpenJDK (como OpenFX) desde 2018. Java FX será o futuro sucessor do Java Swing, incorporando novos templates (*look-and-feel*) e integrando suporte para CSS, embora a Oracle pretenda manter as duas bibliotecas no JSE

Swing e AWT – Componentes

Interfaces Gráficos – Componentes e Gestão de Eventos

Os **componentes generalizados** mantêm as funcionalidades definidas na AWT, **acrescentando-lhes uma série de novas possibilidades**

Como regra geral, a nomeação dos componentes *Swing* é efectuada acrescentando o prefixo “**J**” ao nome do componente AWT correspondente

Exemplo: (AWT)	→ (Swing)
Button	→ JButton
TextFiled	→ JTextField
Applet	→ JApplet

Quanto aos **processadores de eventos**, o *Swing* baseia-se em geral **nos mecanismos fornecidos pela AWT**

No entanto, em virtude da inclusão de novos componentes, **foram definidos também novos eventos e respectivos processadores**

O **API do *Swing***, para além de ser bastante completo e flexível é também extenso, sendo constituído por **18 *packages* públicas**

No entanto, a generalidade dos programas apenas necessitam de **duas *packages***: ***javax.swing*** e ***javax.swing.event*** (esta última apenas nos casos em que se utilizem eventos definidos no *Swing*)

De notar que as *packages* do ***Swing*** têm por base a hierarquia ***javax*** (conjunto de extensões (*java* - *x*) às bibliotecas base do JRE)

Componentes de um Interface Gráfico

Criação do contentor principal (*Content Pane*)

Interfaces Gráficas – Componentes e Gestão de Eventos

Passo 1: A primeira tarefa a realizar na criação de janelas gráficas consiste em **obter o *content pane*** (contentor principal) da janela gráfica, ao qual serão adicionados os diversos componentes que irão constituir a janela gráfica

```
public class Botao extends JFrame {  
    private Container cont;           // Cria objecto contentor  
    //...  
    public Botao() {                  // Construtor  
        cont = getContentPane();     // Inicializa contentor  
        //...  
    }  
    //...  
}
```

Criação do contentor principal (*Content Pane*)

Interfaces Gráficas – Componentes e Gestão de Eventos

Existindo um ***container*** obtido no construtor (invocado no **main** quando for criado um objecto do tipo **Botao** para criar a interface gráfica), e não existindo a intenção de desenhar ou escrever sobre ele, **não será necessário implementar o método `paint()`**, cuja principal função era obter o *container* **Graphics**, para desenharmos a interface pretendida

O **construtor** da classe é assim responsável por definir o *container* principal onde iremos dispor colocar os diversos elementos da interface.

No construtor será possível, por sua vez, **acrescentar *containers* hierarquicamente inferiores** (como **JPanel**) para agrupar conjuntos de elementos

Disposição de elementos no contentor principal

Interfaces Gráficos – Componentes e Gestão de Eventos

Passo 2: Define-se ainda o **modo como os componentes serão organizados no *content pane***. Caso não seja feito, as componentes serão sobrepostas:

```
Container cont = getContentPane();  
cont.setLayout (new FlowLayout());
```

Neste caso, a escolha recaiu sobre o ***flow layout***, que basicamente dispõe os componentes sequencialmente, da esquerda para a direita e de cima para baixo

Disposição de elementos no contentor principal

Interfaces Gráficas – Componentes e Gestão de Eventos

Caso nada seja especificado, é usado o **gestor de posicionamento por defeito**:

- **FlowLayout**, por defeito para *Applet* e *Panel*, onde as componentes são colocados da esquerda para a direita
- **BorderLayout**, por defeito para todos os '*Content Panes*' (*JDialogs*, *JApplets* e *JFrames*), que estabelece 5 zonas para inserção de componentes (Centro, Norte, Sul, Este e Oeste)

Estes e outros *layouts*, como **CardLayout**, **GridLayout** ou **GridBagLayout** formam o grupo de 5 gestores de posicionamento base que serão descritos detalhadamente mais à frente

Elementos gráficos – Adição de elementos ao *content pane*

Seguidamente, há que **criar os objectos necessários (componentes: `JComponent`) e juntá-los ao *content pane***

Passo 3: A **adição de componentes ao *content pane*** é conseguido através do método **`add()`** (a disposição dos vários componentes dependerá do tipo de gestor de posicionamento):

```
Container cont = getContentPane(); // 1 obtém contentor principal
cont.setLayout (new FlowLayout()); // 2 define a disposição
```

```
JButton b1 = new JButton("Botao Um"); // 3a cria elemento gráfico
cont.add(b1); // 3b adiciona elemento gráfico ao contentor principal
```

Interface gráfico - Exemplo

Interfaces Gráficas – Componentes e Gestão de Eventos

Exemplo completo:

```
public class Botao extends JFrame {  
    // Construtor  
    public Botao() {  
        // 1 obtém contentor principal  
        Container cont = getContentPane();  
        // 2 define a disposição  
        cont.setLayout (new FlowLayout());  
  
        // 3a cria elemento gráfico  
        JBUTTON b1 = new JBUTTON("Botao Um");  
        // 3b adiciona elemento gráfico ao contentor principal  
        cont.add(b1);  
  
        //...  
    }  
    //...  
}
```

Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

Os elementos gráficos (**componentes: JComponent**) mais comuns numa interface gráfica são os seguintes :

- Etiquetas (**JLabel**);
- Campos de texto (**TextField** e **PasswordField**);
- Áreas de texto (**TextArea**);
- *Combo boxes* (**ComboBox**);
- Botões (**Button** e **Choice**);
- Botões de selecção (**CheckBox**);
- Grupos de botões de rádio (**RadioButton** e **ButtonGroup**);
- Listas (**List**);
- Tabelas (**Table**);
- Potenciómetros (**Scrollbar** e **Slider**) .

Nota: todas as componentes são subclasses (herdam) de **JComponent**

<http://download.oracle.com/javase/tutorial/uiswing/components/index.html>

<http://java.sun.com/docs/books/tutorial/uiswing/components/index.html>

<http://java.sun.com/docs/books/tutorial/uiswing/examples/components/index.html>

Todos os elementos gráficos herdam da classe **JComponent**, onde estão definidos vários métodos comuns como

- **setEnabled(boolean)** para activar ou desactivar uma componente
- **setVisible(boolean)** para mostrar a componente na interface

Cada componente tem adicionalmente a sua própria API para permitir alterar ou obter os valores manipulados. Por exemplo:

- **campo de texto (JTextField)** para alterar ou obter o respectivo conteúdo disponibiliza os métodos
 - **setText()**
 - **getText()**
- **caixa de combinação (JComboBox)** disponibiliza o método
 - **getSelectedItem()** para obter o elemento seleccionado pelo utilizador, ou
 - **getSelectedValuesList()** implementado pelas listas (**JList**)

Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

Etiquetas (JLabel);



o alinhamento pode se realizado através da classe `SwingConstants` ou através das variáveis `CENTER_ALIGNMENT`, `RIGHT_ALIGNMENT` e `LEFT_ALIGNMENT` :

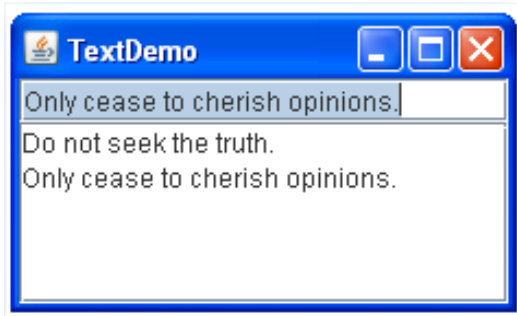
```
JLabel myLabel = new JLabel("Text", (int)LEFT_ALIGNMENT);
```

<http://download.oracle.com/javase/tutorial/uiswing/components/label.html>

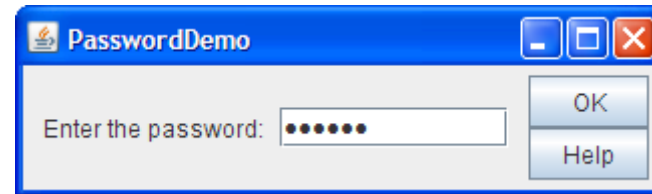
Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

Campos de texto (`JTextField` e `JPasswordField` e `JFormattedTextField`);

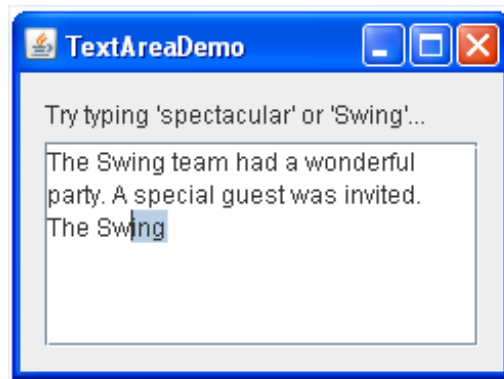


`myTextField.getText()`



`myPasswordField.getPassword()`

Áreas de texto (`JTextArea`);



<http://download.oracle.com/javase/tutorial/uiswing/components/textfield.html>
<http://download.oracle.com/javase/tutorial/uiswing/components/textarea.html>

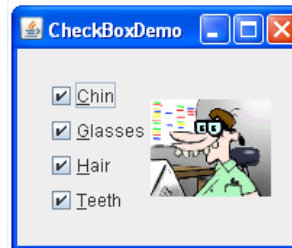
Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

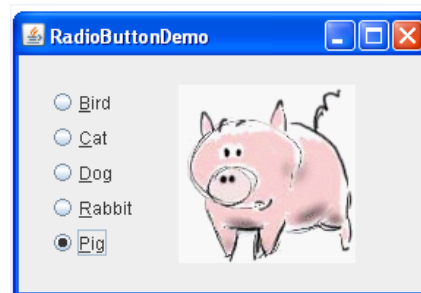
Botões (JButton e JChoice);



Botões de selecção (JCheckBox);



Grupos de botões de rádio (ButtonGroup e JRadioButton);

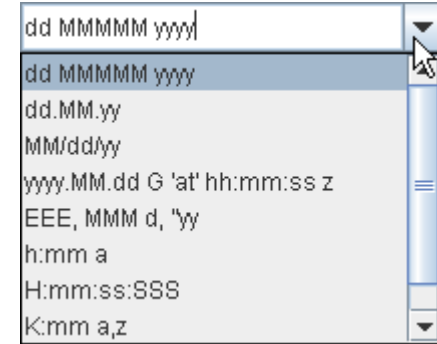
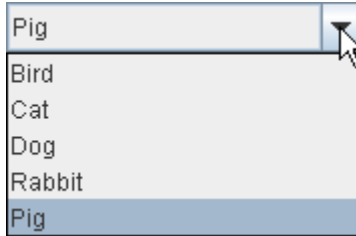


<http://download.oracle.com/javase/tutorial/uiswing/components/button.html>

Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

Caixas de combinação / combo boxes (JComboBox);



```
JComboBox myComboBox = new JComboBox( arrayWithData[] );  
Object obj = myComboBox.getSelectedItem();
```

Para **actualizar *combo boxes*** durante a execução da aplicação, será necessário **associar a componente a um modelo** (DefaultComboBoxModel), tal como se procederá com as tabelas:

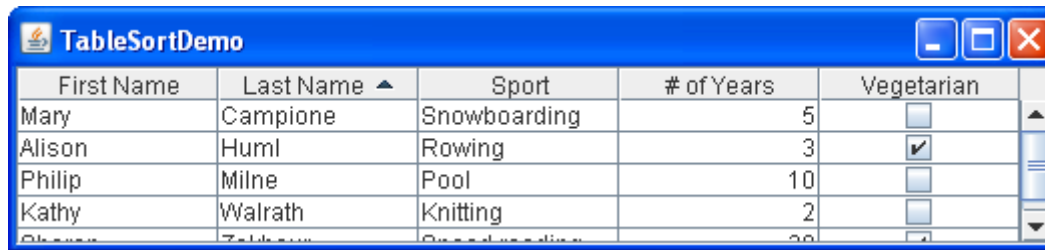
```
JComboBox myComboBox = new JComboBox();  
DefaultComboBoxModel model = new DefaultComboBoxModel( arrayWithData[] );  
myComboBox.setModel(model);  
  
// para actualizar a combo box com novos dados:  
model = new DefaultComboBoxModel( arrayWithNewData[] );  
myComboBox.removeAllItems();  
myComboBox.setModel(model);
```

<http://docs.oracle.com/javase/tutorial/uiswing/components/combobox.html>

Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

As tabelas (`JTable`) permitem visualizar a informação de uma forma formatada



First Name	Last Name ▲	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Philip	Milne	Pool	10	☐
Kathy	Walrath	Knitting	2	☐
Oliver	Zeldman	Good Looking	20	☐

Uma tabela é **inicializada** com a instrução

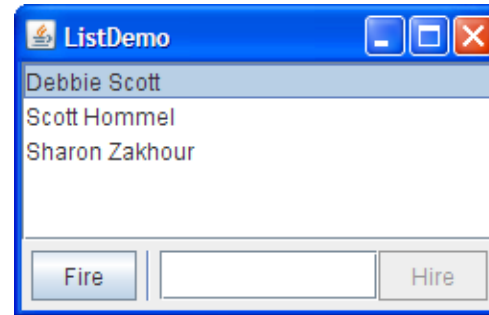
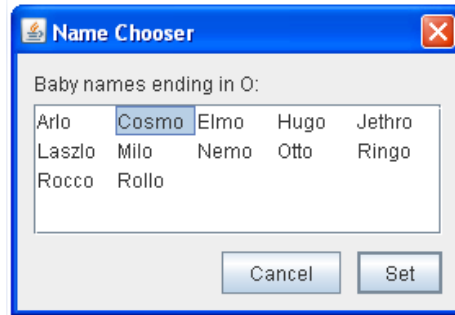
```
JTable table = new JTable(data, columnNames)  
add(table);
```

que recebe como argumento uma **tabela** (ou Vector) '**data**' com os **dados a disponibilizar** e uma **tabela** (ou Vector) '**columnNames**' com o **nome das colunas**

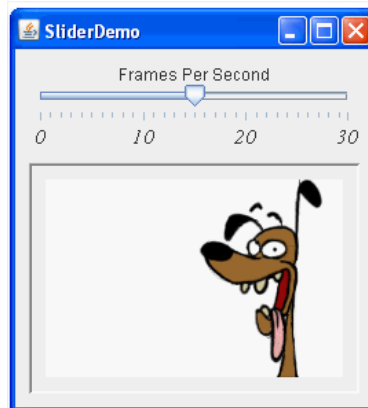
Elementos gráficos

Interfaces Gráficas – Componentes e Gestão de Eventos

Listas (JList);



Potenciómetros (JScrollBar e JSlider) .



<http://download.oracle.com/javase/tutorial/uiswing/components/list.html>
<http://download.oracle.com/javase/tutorial/uiswing/components/slider.html>

Elementos gráficos – Formatação

Interfaces Gráficas – Componentes e Gestão de Eventos

É ainda possível **formatar os diversos elementos** (aplicável a qualquer componente, e.g. `JLabel`, `JTextField`). As formatações mais usuais são a mudança do **estilo** e **cor** de texto, possível através dos métodos

- `setFont()` (mudança do **estilo**)
- `setForeground()` (mudança do **cor**)

Por exemplo:

```
JLabel tag = new JLabel("Texto visível");  
tag.setFont( new Font("Tahoma", Font.PLAIN, 13) );  
tag.setForeground( new Color(0, 0, 254) );
```

No exemplo anterior criou-se uma caixa de texto não editável (`JLabel`), que apresentará o **texto de cor azul** e com o **estilo ‘Tahoma’, regular** (sem negrito ou itálico) e com **tamanho de 13 pixels**

A **definição da cor** é conseguida graças ao uso da classe `Color`, enquanto o **estilo** recorre à classe `Font`, ambos da *package* `java.awt`

<http://java.sun.com/docs/books/tutorial/uiswing/components/index.html>

<http://java.sun.com/docs/books/tutorial/uiswing/examples/components/index.html>

Definição de cores (**setColor**)

Definir a cor a partir de uma paleta de cores pré-definida:

black, blue, cyan, darkGray, gray, green, LightGray, magenta, orange, pink, red, white, yellow

```
setColor (Color.red);           // Há 13 cores pré-definidas
```

Definir a cor recorrendo ao método RGB:

```
Color c  = new Color(R, G, B) ;           // Red, Green, Blue
```

```
Color c1 = new Color(0, 255, 0) ;           // verde
```

```
Color c2 = new Color(117, 23, 48) ;         // castanho
```

```
setColor (c1);                     // usando um objecto do tipo Color
```

```
setColor (new Color(255, 0, 0)); // recorrendo ao método RGB
```

Definição de fontes

É possível **modificar os parâmetros da fonte** (classe **Font**) para obter um texto formatado pretendido.

Para o efeito é necessário **invocar um objecto de uma classe **Font****, definido na *package* **java.awt**, definindo três argumento: **nome da fonte**, **estilo da fonte** e **tamanho da fonte**, com a seguinte a sua sintaxe:

```
Font f = new Font(    "Nome_da_Fonte",  
                      Estilo_da_Fonte,  
                      Tamanho  
                      );
```

A seguir à declaração do objecto da classe **Font** é necessário utilizar um método denominado **setFont()**, para esta **ser aplicada ao objecto da classe **Graphics**** que estiver a ser utilizado

```
g.setFont(f);
```

Definição de fontes

O “**Nome da Fonte**” é uma *String* contendo o nome pelo qual a fonte é conhecida. Algumas das **fontes disponíveis** são as seguintes:

- “**TimesRoman**”,
- “**Courier**”,
- “**Terminal**”,
- “**Helvetica**”,
- “**Dialog**” e
- “**Σψμβολ**” (Symbol)

O “**Estilo da Fonte**” é uma variável que representa os tipos de **formatação de texto adicionais** que gostamos de dar aos nossos textos, por exemplo, realçando as palavras a **negrito** (*Bold*) ou a *Itálico*. A lista completa está definida na classe `Font`, mas os estilos mais usuais são o

- “**Font.PLAIN**”,
- “**Font.BOLD**”,
- “**Font.ITALIC**”

O “**Tamanho**” indica o número de pontos que devem representar a letra

Elementos gráficos – Formatação

Interfaces Gráficos – Componentes e Gestão de Eventos

Como vimos anteriormente, a **formatação do texto de *labels*** pode ser conseguida através dos **métodos de formatação** disponibilizados pela classe **JLabel**:

```
JLabel labelScaleMin = new JLabel();  
labelScaleMin.setHorizontalAlignment(JLabel.RIGHT);  
labelScaleMin.setSize(100, 10);  
labelScaleMin.setForeground(new Color(150,150,150));
```

Alternativamente, pode-se aplicar **tags HTML** ao texto para realizar a formatação pretendida:

```
int scaleMin = 10;  
labelScaleMin.setText("<html> <font color='blue' size=\"2\" >"+  
    scaleMin +  
    "</font></html>" );  
  
labelScaleMin.setText("<html><font color=\"8b8989\" size=\"2\" >"+  
    scaleMin +  
    "</font></html>" );
```

Elementos gráficos – Alinhamento

Interfaces Gráficas – Componentes e Gestão de Eventos

Existe um **procedimento de alinhamento** comum a todos as componentes (e.g. **JLabel**, **JTextField**) através dos métodos

- **setHorizontalAlignment()** e
- **setVerticalAlignment()**.

Por exemplo, para uma **JLabel**:

```
JLabel myLabel = new JLabel("Text");
```

```
myLabel.setHorizontalAlignment( javax.swing.SwingConstants.RIGHT );
```

OU

```
myLabel.setHorizontalAlignment( JLabel.RIGHT );
```

E para o caso de outra componente, como uma **JTextField**:

```
JTextField myField = new JTextField (5);
```

```
myField.setHorizontalAlignment( javax.swing.SwingConstants.RIGHT );
```

Elementos gráficos – Dimensionamento

Interfaces Gráficas – Componentes e Gestão de Eventos

As componentes podem ser geradas com um tamanho pré-definido, através do método **setPreferredSize()**, que recebe um objecto **Dimension** e define a dimensão da componente (**comprimento, altura**):

```
JButton myButton = new JButton("text");
```

```
myButton.setPreferredSize( new Dimensions(200,50) );
```

De notar que não é usado o método **setSize()** – que também recebe como argumento um objecto **Dimension**

O método **setSize** é usado para (re-)definir a dimensão de uma componente depois da sua criação

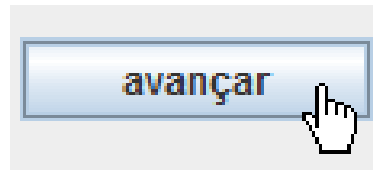
Elementos gráficos – Modificação do estado do cursor

Interfaces Gráficas – Componentes e Gestão de Eventos

Uma forma de auxiliar o utilizador na interacção com a interface gráfica baseia-se na **modificação das imagens do cursor**, por exemplo, indicando que a zona serve para ser pressionada ou para introdução de texto.

Esta acção é conseguida através do método **setCursor()** que recebe como argumento um objecto **Cursor**, que define o tipo de imagem do cursor a apresentar:

```
Jbutton myButton = new JButton("avançar");  
myButton.setCursor(new Cursor(Cursor.HAND_CURSOR));
```



Outras propriedades comuns dos cursores:

CROSSHAIR_CURSOR,
HAND_CURSOR,
TEXT_CURSOR,

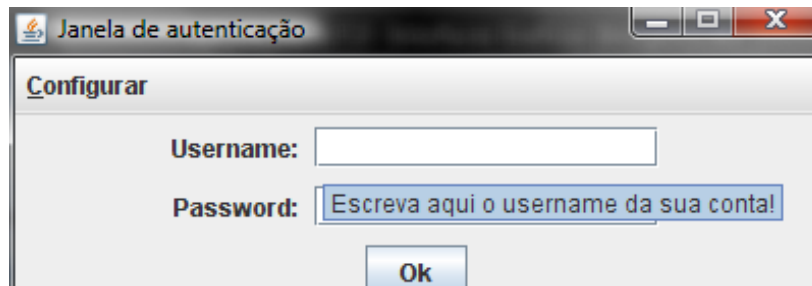
CUSTOM_CURSOR,
MOVE_CURSOR,
WAIT_CURSOR

Elementos gráficos – Mensagem informativa (*tool tip*)

Interfaces Gráficas – Componentes e Gestão de Eventos

É possível associar às diferentes componentes de um interface gráfico (como as `JTextField`) uma **caixa de diálogo informativa** (*Tool Tip*), visualizada sempre que o rato sobrepõe a componente.

Para o efeito recorre-se ao método `setToolTipText("Texto")` ;



- Definir o **tempo de espera** até ao aparecimento da *tool tip* (`ToolTipManager.sharedInstance().setInitialDelay( milliseconds )`),
- Definir o **tempo de apresentação** da *tool tip* ao utilizador (`ToolTipManager.sharedInstance().setDismissDelay( milliseconds )`)

De notar que a propriedade `javax.swing.ToolTipManager` afecta todas as componentes *swing* da aplicação

Elementos gráficos: Limitação de caracteres em campos de texto

É possível restringir o número de caracteres visíveis em cada campo de texto através de validação

A primeira abordagem implica **associar um evento `ActionListener` ao campo de texto, verificando o comprimento do texto sempre que o utilizador pressiona uma tecla**, mantendo o número de caracteres limitado

Uma abordagem alternativa baseia-se no uso da propriedade **`setDocument()`** que implica implementar uma validação através da interface **`DocumentFilter`** OU **`PlainDocument`**

Por fim, é possível **reescrever a classe `JTextField`** para adicionar um parâmetro adicional (comprimento máximo), controlado através de uma das interfaces **`DocumentFilter`** OU **`PlainDocument`**

Elementos gráficos: Limitação de caracteres em campos de texto

Interfaces Gráficas – Componentes e Gestão de Eventos

```
import javax.swing.text.PlainDocument;

public class JTextFieldLimit extends PlainDocument {
    private int limit;

    JTextFieldLimit(int limit) {
        super();
        this.limit = limit;
    }

    // método responsável pela validação
    public void insertString( int offset, String str, AttributeSet attr )
        throws BadLocationException {

        if (str == null) return;
        if ((getLength() + str.length()) <= limit) {
            super.insertString(offset, str, attr);
        }
    }
}
```

Tendo sido criada a classe responsável, podemos associar essa propriedade à tua caixa de texto:

```
// tamanho visual da caixa de texto (número de colunas)
JTextField textfield = new JTextField(10);
textfield.setDocument( new JTextFieldLimit(5) );
// número máximo de caracteres, usando a classe criada anteriormente (JTextFieldLimit)
para esse efeito.
```

Elementos gráficos: Limitação de caracteres em campos de texto

Interfaces Gráficas – Componentes e Gestão de Eventos

Implementação alternativa usando a interface **DocumentFilter**:

```
public class LengthFilter extends DocumentFilter {
    private int max;

    public LengthFilter(int maxLength) {
        max = maxLength;
    }

    public void insertString(DocumentFilter.FilterBypass fb, int offset,
                             String text, AttributeSet attr) throws BadLocationException {
        if (fb.getDocument().getLength() + text.length() <= max)
            fb.insertString(offset, text, attr);
        else
            Toolkit.getDefaultToolkit().beep();
    }

    public void replace(DocumentFilter.FilterBypass fb, int offset,
                        int length, String text, AttributeSet attr) throws BadLocationException {
        if (fb.getDocument().getLength() + text.length() - length <= max)
            fb.replace(offset, length, text, attr);
        else
            Toolkit.getDefaultToolkit().beep();
    }
}
```


Elementos gráficos – Transparência dos objectos

Interfaces Gráficas – Componentes e Gestão de Eventos

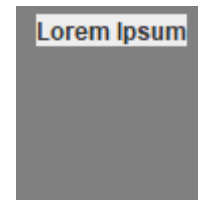
Se pretendermos que o **fundo** (*background*) de uma componente fique **transparente** para que se possa observar as componentes subjacentes podemos usar a propriedade **setOpaque(False)**

Esta é uma situação **comum em labels** onde se pretende que se veja **apenas o texto (e não o fundo da label)** bem como a imagem associada ao painel onde a *label* está integrada (integrar imagens em painéis ou outras componentes será estudado na secção seguinte)

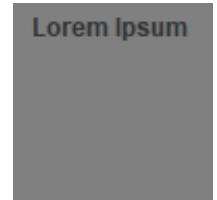
```
JLabel myLabel = new JLabel("Lorem Ipsum");  
myLabel.setOpaque(false);
```

```
JPanel myPanel = new JPanel();  
myPanel.add(myLabel);
```

Opaque = true



Opaque = false



Desta forma será **visível o texto da label** bem como o fundo do painel onde esta se encontra inserida, **não sendo visível o fundo da label**

Gestão de Eventos

Para uma aplicação **manipular eventos**, é necessário o código contemplar os seguintes procedimentos:

- 1 – Indicar **que classes contêm componentes que podem gerar eventos** como reacção a uma acção do utilizador;
- 2 – Indicar as **componentes que podem gerar eventos** e que objectos são responsáveis pelo seu tratamento;
- 3 – Implementar as **acções a realizar após um evento ser gerado**.

Acções sobre os elementos – gestão de eventos

Interfaces Gráficas – Componentes e Gestão de Eventos

As interfaces gráficas **devem reagir a uma acção** do utilizador, por exemplo, quando um utilizador pressiona um botão

Passo 1: Para existir **gestão de eventos** a classe deve implementar a interface respectiva, por exemplo a interface **ActionListener**:

```
public class MinhaClasse extends JFrame
                                implements ActionListener {
    //...
}
```

Quando uma **componente da interface** é activada (e.g. um botão é pressionado) dizemos que **ocorre um evento** e em termos internos é **gerado um evento** (objecto) da classe adequada (neste caso particular da classe **ActionEvent**)

De realçar a existência de diversas interfaces de gestão de eventos para diferentes fins (e.g. **ActionListener**, **MouseListener**, **KeyListener**)

Passo 2: É necessário indicar que componentes podem **gerar eventos** e que objectos serão responsáveis por **tratar esses eventos**

Objectos que respondem a eventos chamam-se '**event listeners**' ou **processadores de eventos**

Para o efeito usámos o método **addActionListener()**

```
Jbutton b1 = new Jbutton("First button");  
b1.addActionListener(this);
```

Estes eventos têm que ser notificados a algo que inicie as acções correspondentes

Note no exemplo anterior o uso de **this** para **referenciar a própria classe** como '**event listener**' do botão, ou seja, o objecto que aguarda a geração de eventos e realiza o seu tratamento é a própria classe

Na realidade, `ActionListener` é uma **interface** a partir da qual podemos definir classes de objectos '***event listeners***'

Repare que a classe da interface gráfica implementa (**`implements`**) essa **interface**

Passo 3: É necessário indicar **que acções serão realizadas após a ocorrência de um evento** (tratamento do evento, ou reacção da aplicação à acção do utilizador sobre a interface)

A interface ***ActionListener*** possui o método ***actionPerformed()*** que é **invocado automaticamente quando ocorre um evento** do tipo que a ***ActionListener*** detecta (***ActionEvent***)

Uma classe, ao implementar a interface ***ActionListener*** (***implements ActionListener***), terá obrigatoriamente que reescrever e **implementar o método *actionPerformed()***:

```
public void actionPerformed (ActionEvent e) {  
    if (e.getSource().equals(b1)) // Determina o botão premido  
        label.setText("button 1 pressed: " + b1.getText());  
}
```

Resumindo, o tratamento de eventos requer três blocos de instruções:

1. Na declaração da classe onde irá ocorrer a resposta ao evento, **deve ser indicado que a classe implementa um interface *listener***:

```
public class MinhaClasse extends JFrame
    implements ActionListener { /*...*/ }
```

2. O **registo da classe responsável pelo processamento do evento** (neste caso a classe `MinhaClasse`) é realizado pelo método **`addActionListener`** sobre os componentes que poderão gerar o evento:

```
b1.addActionListener(this);
```

3. **A classe que faz o processamento do evento deve implementar o(s) método(s) (neste caso o `actionPerformed()`) do interface *listener* (`ActionListener`) em causa:**

```
public void actionPerformed (ActionEvent e) {
    // instruções que serão executadas quando ocorrer o evento
}
```


Para **impedir a repetição de código** e consumo de recursos, **o tratamento de eventos gerados pelas componentes da interface deve ser realizado num único método** que centraliza os eventos gerados. Neste caso uma classe possui um único método `actionPerformed` (para eventos do *listener* `ActionListener`)

Porém, é possível **aplicar o código de tratamento de evento de uma componente na definição da própria componente**, através de uma classe anónima (interna):

```
Jbutton b1 = new Jbutton("name of button");
b1.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            // instruções que serão executadas quando ocorrer o evento
            label.setText("button 1 pressed: " + b1.getText());
        }
    });
```

Esta solução é utilizada quando **cada componente tem um evento único** e a interface é composta por um **número reduzido de componentes** que geram eventos. Nesta solução, a classe principal não necessita de implementar o *listener*

O mecanismo anterior pode ser adaptado recorrendo à classe **AbstractAction** (que implementa a interface **ActionListener**) e evitando a criação de objectos para cada componente:

```
JPanel panel = new JPanel();
panel.add(
    new JButton(
        new AbstractAction("name of button") {
            public void actionPerformed(ActionEvent e) {
                // instruções que serão executadas quando ocorrer o evento
            }
        }
    );
```

De realçar que nesta solução **deixa de se ter acesso à componente** (botão), uma vez que não é criado um objecto que possa ser acedido e passado como parâmetro entre métodos.

Apesar de ser uma forma contraída (recorrendo a classes anónimas internas) que localiza os eventos na definição das componentes, **não é possível durante execução conhecer ou alterar o estado da componente**, uma vez que não existe uma referência (objecto)

Interface *Listener*

Há uma interface *listener* definida para cada tipo de evento, contendo cada uma delas os métodos necessários para responder a esse tipo de evento

Uma classe *listener* implementa uma dada interface *listener*, pelo que deve implementar todos os métodos nela definidos

Os *listeners* são adicionados aos componentes

Quando um componente gera um evento, o método correspondente a esse evento é executado no seu *listener*

Um componente pode ter vários *listeners* (para vários eventos)

Uma classe *listener* pode implementar vários interfaces *listener*, ou seja pode detectar vários tipos de eventos

Interfaces *Listener* e eventos

Interfaces Gráficos – Componentes e Gestão de Eventos

Existem diversos tipos de **eventos** e cada um deles tem o seu tipo de **listener** que terá que ser implementado

Por exemplo:

Acção do utilizador	Evento gerado	<i>Event listener</i>
Seleccionar botão	<code>ActionEvent</code>	<code>ActionListener</code>
Mover o rato	<code>MouseEvent</code>	<code>MouseListener</code>
Rato entra no componente	<code>FocusEvent</code>	<code>FocusListener</code>
Escrever no teclado	<code>KeyEvent</code>	<code>KeyListener</code>
Escrever num <i>TextField</i>	<code>ActionEvent</code>	<code>ActionListener</code>

Todos os eventos pertencem à package **`java.awt.event`** e herdam da classe (`java.awt`) **`AWTEvent`**

Interfaces *MouseListener* e *MouseMotionListener*

Interfaces Gráficos – Componentes e Gestão de Eventos

A interface **MouseListener**, **gere as acções sobre o rato**, permitindo verificar que botões do rato foram pressionados e se o rato está a interagir com alguma componente da interface, através dos métodos

mousePressed(MouseEvent e)

mouseReleased(MouseEvent e)

mouseEntered(MouseEvent e)

mouseExited(MouseEvent e)

mouseClicked(MouseEvent e)

A par da interface **MouseListener**, existe a interface **MouseMotionListener** que **permite acompanhar o movimento do rato**, e não apenas quando este executa uma acção específica sobre uma componente. Esta interface implica a implementação dos métodos

mouseDragged(MouseEvent e)

mouseMoved(MouseEvent e)

Todos os métodos recebem eventos do tipo `java.awt.event.MouseEvent`

Interface *MouseListener*

Interfaces Gráficos – Componentes e Gestão de Eventos

Quando se manipula um rato é por vezes **necessário conhecer qual botão foi pressionado** (esquerdo, direito ou central)

Para obter essa informação podemos recorrer às propriedades da interface **MouseListener** OU da classe **SwingUtilities**:

```
public class MyClass implements MouseListener {  
    // Code  
  
    public void mouseClicked(MouseEvent event) {  
        int mX = (int) event.getPoint().getX();  
        int mY = (int) event.getPoint().getY();  
  
        if (event.getButton() == MouseEvent.BUTTON1) {  
            // acções associadas ao clique no botão esquerdo  
        }  
  
        if (event.getButton() == MouseEvent.BUTTON2) {  
            // acções associadas ao clique no botão do centro  
        }  
  
        if (event.getButton() == MouseEvent.BUTTON3) {  
            // acções associadas ao clique no botão direito  
        }  
    }  
}
```

Interface *MouseListener*

Interfaces Gráficos – Componentes e Gestão de Eventos

Com a classe **SwingUtilities** podemos verificar 3 propriedades:

SwingUtilities.isLeftMouseButton(MouseEvent anEvent)

SwingUtilities.isRightMouseButton(MouseEvent anEvent)

SwingUtilities.isMiddleMouseButton(MouseEvent anEvent)

Por exemplo:

```
public class MyClass implements MouseListener {
    // Code

    public void mouseClicked(MouseEvent event) {

        if (SwingUtilities.isRightMouseButton(event)) {
            // acções associadas ao clique no botão direito
            // equivale a event.getButton() == MouseEvent.BUTTON3
        }

    }
}
```

Interfaces *KeyListener* e *FocusListener*

Interfaces Gráficos – Componentes e Gestão de Eventos

A interface **KeyListener** implementa os eventos `java.awt.event.KeyEvent`, disponibilizando os métodos

```
keyPressed(KeyEvent e),  
keyReleased(KeyEvent event), e  
keyTyped(KeyEvent e)
```

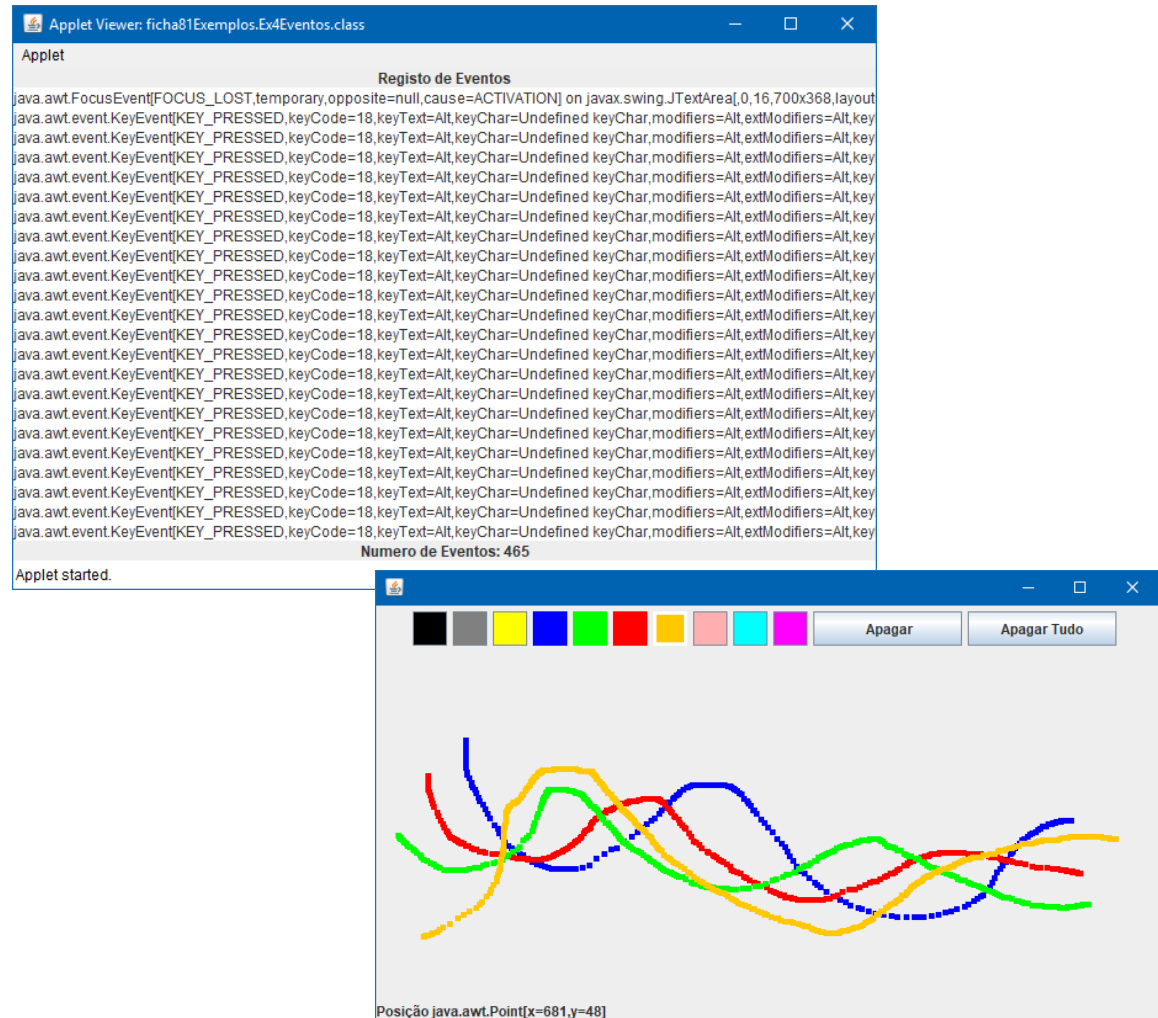
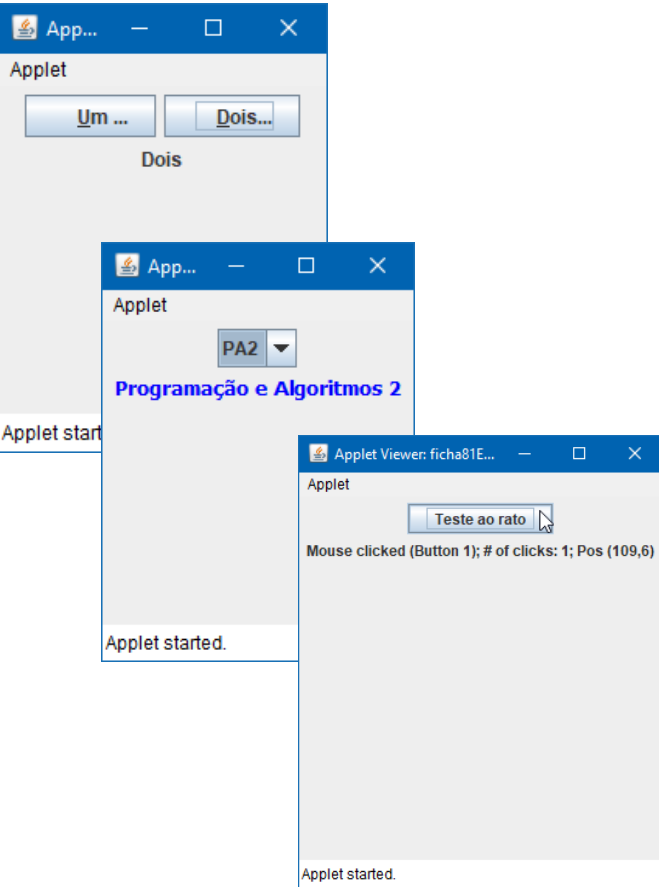
para o tratamento de eventos gerados pelo teclado

A interface **FocusListener** implementa os eventos `java.awt.event.FocusEvent`, disponibilizando os métodos

```
focusGained(FocusEvent event), e  
focusLost(FocusEvent event)
```


Exemplo de interfaces em Java Swing

Interfaces Gráficas – Componentes e Gestão de Eventos



“Programação Orientada a Objectos”

António José Mendes

Departamento de Engenharia Informática, Universidade de Coimbra

“Java Swing”, 2ª Edição

James Elliott, Robert Eckstein (Editor), Marc Loy, David Wood, Brian Cole
O'Reilly, ISBN: 0596004087

“Thinking in Java”, 4ª Edição, Capítulo 14 “Creating Windows & Applets”

Bruce Eckel

Prentice Hall, ISBN: 0131872486

“The Java Tutorial – Creating a GUI with JFC/Swing”

Java Sun Microsystems

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino